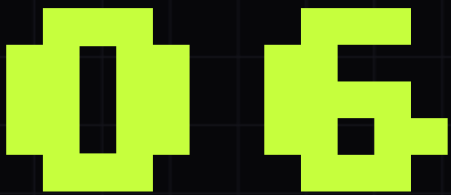


MODULE 06 OF 09



Data & Progression

YOU'LL SHIP

Persistent player profiles with coins, XP and levels.

- 6.1 Leaderstats: coins and points
- 6.2 DataStoreService
- 6.3 Saving on leave and on server shutdown
- 6.4 XP and levels

Players come back to games that remember them. In this module you'll give every player a coin balance, save it with DataStores so it survives leaving and server shutdowns, and add XP and levels so there's always a next goal.

6.1 Leaderstats: coins and points

Roblox has a built-in leaderboard. If a player has a folder named exactly `leaderstats` with value objects inside, they show up in the top-right player list automatically.

Script in **ServerScriptService** named `PlayerData` :

```
local Players = game:GetService("Players")

Players.PlayerAdded:Connect(function(player)
    local leaderstats = Instance.new("Folder")
    leaderstats.Name = "leaderstats"
    leaderstats.Parent = player

    local coins = Instance.new("IntValue")
    coins.Name = "Coins"
    coins.Value = 0
    coins.Parent = leaderstats

    local level = Instance.new("IntValue")
    level.Name = "Level"
    level.Value = 1
    level.Parent = leaderstats
end)
```

Press Play: your name appears with Coins and Level. The HUD from Module 4 now shows your coin count too.

WATCH OUT Only the **server** should change these values. If a LocalScript changes them, only that player sees the change, and it won't save.

6.2 DataStoreService

A **DataStore** is a cloud database Roblox gives every game for free. You save a value under a key (like the player's UserId) and load it next time.

First, allow Studio to use it: **File** → **Game Settings** → **Security** → **Enable Studio Access to API Services**. The place must be published.

DataStore calls go over the network and **can fail**, so every call is wrapped in `pcall` :

```
local DataStoreService = game:GetService("DataStoreService")
local store = DataStoreService:GetDataStore("PlayerData_v1")

local ok, data = pcall(function()
    return store:GetAsync("Player_" .. player.UserId)
end)

if ok then
    -- data is the saved table, or nil for a brand-new player
else
    warn("Load failed:", data)
end
```

Save a **table** per player, not separate keys per stat. One read and one write per player is faster and stays within limits:

```
local DEFAULT_DATA = {
    Coins = 0,
    XP = 0,
    Level = 1,
    Upgrades = {},
}
```

6.3 Saving on leave and on server shutdown

Here's the complete, production-style `PlayerData` script. It loads with retries, keeps a session copy in memory, saves when a player leaves, autosaves every few minutes, and saves everyone when the server shuts down.

```

local Players = game:GetService("Players")
local DataStoreService = game:GetService("DataStoreService")

local store = DataStoreService:GetDataStore("PlayerData_v1")

local DEFAULT_DATA = { Coins = 0, XP = 0, Level = 1, Upgrades = {} }
local AUTOSAVE_INTERVAL = 180

local sessionData: { [Player]: any } = {}

local function keyFor(player: Player): string
    return "Player_" .. player.UserId
end

local function withRetries<T>(fn: () -> T, attempts: number): (boolean, T?)
    for attempt = 1, attempts do
        local ok, result = pcall(fn)
        if ok then return true, result end
        warn(`DataStore attempt {attempt} failed: {result}`)
        task.wait(2 ^ attempt)
    end
    return false, nil
end

local function load(player: Player)
    local ok, saved = withRetries(function()
        return store:GetAsync(keyFor(player))
    end, 3)

    if not ok then
        -- Don't let them play with a blank profile that would overwrite real data.
        player:Kick("Couldn't load your data. Please rejoin in a minute.")
        return
    end

    local data = table.clone(DEFAULT_DATA)
    data.Upgrades = {}
    if type(saved) == "table" then
        for k, v in saved do data[k] = v end
    end
    sessionData[player] = data

    local leaderstats = Instance.new("Folder")

```

```

    leaderstats.Name = "leaderstats"
    local coins = Instance.new("IntValue")
    coins.Name = "Coins"
    coins.Value = data.Coins
    coins.Parent = leaderstats
    local level = Instance.new("IntValue")
    level.Name = "Level"
    level.Value = data.Level
    level.Parent = leaderstats
    leaderstats.Parent = player

    -- Keep the session copy in sync with the visible values.
    coins.Changed:Connect(function(v) data.Coins = v end)
    level.Changed:Connect(function(v) data.Level = v end)
end

local function save(player: Player)
    local data = sessionData[player]
    if not data then return end
    withRetries(function()
        return store.UpdateAsync(keyFor(player), function()
            return data
        end)
    end, 3)
end

Players.PlayerAdded:Connect(load)
for _, p in Players:GetPlayers() do task.spawn(load, p) end

Players.PlayerRemoving:Connect(function(player)
    save(player)
    sessionData[player] = nil
end)

-- Autosave
task.spawn(function()
    while true do
        task.wait(AUTOSAVE_INTERVAL)
        for player in sessionData do
            task.spawn(save, player)
        end
    end
end)
end)

```

```
-- Server shutdown: save everyone before the server closes (max ~30 s)
game:BindToClose(function()
    local pending = 0
    for player in sessionData do
        pending += 1
        task.spawn(function()
            save(player)
            pending -= 1
        end)
    end
    while pending > 0 do task.wait() end
end)
```

Why each part matters:

- **Retries with backoff:** DataStores occasionally fail; waiting 2, 4, 8 seconds and retrying fixes most failures.
- **Kick on failed load:** if you let a player in with default data, the next save overwrites their real progress. Kicking is annoying, but losing progress is far worse.
- `UpdateAsync` instead of `SetAsync` : safer when several servers might write the same key.
- `BindToClose` : when Roblox shuts a server down (for example after you publish an update), players don't get `PlayerRemoving` in time without it.

TIP Changing the store name (`PlayerData_v1` → `PlayerData_v2`) gives everyone a fresh start. Use it for testing, never casually in a live game.



XP and levels

Levels give players a reason to play one more round. Use a formula so each level needs a bit more XP than the last:

```
local function xpForLevel(level: number): number
    return math.floor(50 * level ^ 1.5)
end

-- Level 1→2: 50 XP, 2→3: 141, 5→6: 559, 10→11: 1581
```

A server function to award XP, used by the round system in Module 7:

```

local LevelUp = game:GetService("ReplicatedStorage").Remotes.LevelUp -- RemoteEvent

local function addXP(player: Player, amount: number)
    local data = sessionData[player]
    if not data then return end
    data.XP += amount
    while data.XP >= xpForLevel(data.Level) do
        data.XP -= xpForLevel(data.Level)
        data.Level += 1
        player.leaderstats.Level.Value = data.Level
        LevelUp:FireClient(player, data.Level)
    end
end
end

```

On the client, listen to `LevelUp` and play a big celebratory UI animation, a sound, and a particle burst. That moment of feedback is what makes progression feel good.

Sharing functions between scripts. Other server scripts (the round system, the shop) need `addXP` and access to coins. Turn the data code into a **ModuleScript** `PlayerDataService` in `ServerScriptService` that returns a table with `get(player)`, `addCoins(player, n)` and `addXP(player, n)`, and have one Script `require` it on startup. That's the structure Module 7 uses.

TRY IT

1. Collect a few coins, stop the test, press Play again: your coins should still be there.
2. Add a `TotalCoinsEver` stat that only goes up, and show it in a stats panel.

Module recap

- ☐ leaderstats show Coins and Level
- ☐ Data loads with retries and kicks on failure instead of overwriting
- ☐ Saves on leave, every 3 minutes, and on server shutdown
- ☐ XP curve and level-ups with a client celebration